

“A File Versioning and Recovery System”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

P NEEHARIKA	2520090233
CHAKRIKA	2520090044
G SHIVANI	2520090046

Under the Esteemed Guidance of

[Guide Name]

Assistant Professor

Department of Computer Science and Information Technology



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled “**A File Versioning and Recovery System**” is a record of bonafide work of **P NEEHARIKA – 2520090233, CHAKRIKA – 2520090044, G SHIVANI – 2520090046** submitted in partial fulfillment of the requirements of the course DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E), Section 10, Team 35, during 2026–2027, Trimester 4, at KLH University. The results embodied in this report have not been copied from any other student, department, or institution.

P NEEHARIKA – 2520090233

CHAKRIKA – 2520090044

G SHIVANI – 2520090046

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the project report entitled “**A File Versioning and Recovery System**” is a bonafide work done and submitted by **P NEEHARIKA – 2520090233, CHAKRIKA – 2520090044, G SHIVANI – 2520090046** in partial fulfillment of the requirements of the course DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E), Section 10, Team 35, at KLH University during 2026–2027, Trimester 4.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who have assisted us in one way or the other for the completion of our project.

Our greatest appreciation to our Course Coordinator [Course Coordinator Name], and our guide [Guide Name], Department of Computer Science and Information Technology, which cannot be expressed in words, for their tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. Kaja Shareef**, Head of the *Department of Computer Science and Information Technology* for providing us with adequate facilities, ways and means by which we were able to complete this project-based Lab.

We thank all the members of the teaching and non-teaching staff who have assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

P NEEHARIKA – 2520090233

CHAKRIKA – 2520090044

G SHIVANI – 2520090046

Abstract

The project “A File Versioning and Recovery System” is designed to develop a simple and organized system for managing different versions of files. The main concept of the project is to maintain a history of a file as it is created and modified. Instead of replacing the previous version whenever a file is updated, the system stores the new version separately along with relevant details such as the version number, file name, and date and time of modification. This allows users to view and manage the different versions of a file through a structured version history.

The system provides features for creating new versions, viewing available versions, and maintaining the stored file history. Users can select a particular version and view its details, making it easier to keep track of changes made to a file over time. The system also supports comparison between different versions, allowing users to identify the changes made between them. These features are provided through a simple and easy-to-use web interface.

Another important part of the project is the recovery mechanism, which allows users to restore an earlier version of a file when required. If a user makes unwanted changes, replaces a file, or accidentally deletes it, an available previous version can be selected and restored, and every restoration is recorded in a recovery log. The application uses HTML, CSS and JavaScript for the frontend, Java for the backend logic, a MySQL relational database for storing file information, version details, timestamps, file paths and recovery status, and local file storage for the uploaded files and their versions. Overall, the project combines version tracking and file recovery into a single system and provides a practical implementation of version control concepts in a general file management environment.

Index

S.No.	Chapters	Topics	Page No.
		Acknowledgement	
		Abstract	
1	Introduction	1.1 Background of the Project 1.2 Problem Statement 1.3 Scope of the Project 1.4 Objectives 1.5 Importance of the Application 1.6 Target Users / Audience	5
2	System Requirements	2.1 Hardware Requirements 2.2 Software Requirements 2.3 Development Tools and Frameworks	7
3	Technology Stack	3.1 Frontend 3.2 Backend 3.3 Database 3.4 Version Control 3.5 Authentication and Security Libraries	8
4	System Architecture	4.1 High-Level Architecture Diagram 4.2 Description of Each Layer 4.3 Deployment Architecture	9
5	Design	5.1 Database Design Principles 5.2 ER Diagram / Database Schema 5.3 Design Decisions and Assumptions	11
6	Implementation	6.1 Module-wise Implementation 6.2 Front-end Logic 6.3 API Integration 6.4 Authentication and Authorization	13
7	Features	7.1 List of Main Features 7.2 How Each Feature Works	15
8	Testing	8.1 Postman API Testing 8.2 Integration Testing 8.3 Tools Used for Testing 8.4 Test Cases and Results	17
9	Deployment	9.1 Steps to Deploy 9.2 Environment Configuration 9.3 Hosting of Frontend and Backend	19
10	Challenges & Limitations	10.1 Issues Faced During Development 10.2 Solutions Applied 10.3 Current Limitations or Known Bugs	20

S.No.	Chapters	Topics	Page No.
11	Future Enhancements	11.1 Planned Features 11.2 Possible Integrations or Optimizations	21
12	Conclusion	12.1 Summary of the Project 12.2 What Was Achieved 12.3 Skills Learned	22
13	References	– Documentation, tools, and resources used	23
14	Appendices	– Sample Code Snippets – Installation / Setup Instructions – Default Login Credentials – API Reference Summary – Screenshots (to be added by the student)	24

CHAPTER 1: INTRODUCTION

1.1 Background of the Project

Files are constantly created, edited, replaced and deleted during everyday work on documents, reports, source code, spreadsheets and presentations. In most operating systems, saving a changed file simply overwrites the earlier content. Once the new content has been written, the previous version is gone, and an accidental edit, a corrupted save or an unintended deletion can result in permanent data loss unless the user has been maintaining separate backup copies by hand.

Developer-oriented version control tools such as Git solve this problem well, but they are built around commits, branches and command-line workflows that are too complex for general file management and for non-developer users. There is a need for a lightweight and transparent system that tracks file versions automatically and lets users recover an earlier version through a simple graphical interface.

A File Versioning and Recovery System was developed to fill this gap. Instead of replacing a file when it is updated, the system stores each new version separately together with its version number, file name, size and date and time of modification. The metadata is kept in a MySQL database and the physical files are kept in local storage, so the complete history of every file can be viewed, compared and restored at any time. Table 1.1 compares the proposed system with the two approaches it is positioned between.

System Attribute	Traditional OS File Systems	Developer VCS (Git)	Proposed Versioning System
Overwrite Prevention	No (replaces file)	Yes (commit-based)	Yes (automated snapshotting)
User Accessibility	High (basic GUI)	Low (CLI / complex workflow)	High (intuitive web GUI)
Metadata Indexing	Basic (last modified)	Commit trees and logs	Structured MySQL metadata
Recovery Mechanism	Trash / manual backups	Revert / checkout commands	One-click version restore

Table 1.1: Comparison of the proposed system with existing approaches

1.2 Problem Statement

Standard operating-system file managers replace the old file whenever a file is saved, so accidental updates, file corruption or unintended deletion lead to permanent loss of data. Full version control systems, on the other hand, are too complex for general users. The problem addressed by this project is therefore to design a system that (i) preserves every version of a file instead of overwriting it, (ii) maintains a structured, searchable history of those versions, (iii) allows two versions to be compared to identify what changed, and (iv) allows any earlier version to be restored when unwanted changes are made or a file is deleted by mistake.

1.3 Scope of the Project

The system covers the lifecycle of a file from the version-tracking point of view: uploading a

new file, uploading an updated copy of an existing file, viewing file details and version history, comparing two versions, downloading a version, restoring an earlier version, archiving a file instead of permanently deleting it, and recording each recovery in an audit log. The actual files are stored on the local file system, and their metadata (version number, path, size, timestamps and status) is stored in MySQL. Cloud storage, delta compression of versions and multi-user collaborative editing are outside the scope of this submission and are listed under future enhancements.

1.4 Objectives

- Store every upload or update as a separate version instead of replacing the existing file.
- Assign a sequential version number (v1.0, v1.1, v1.2 and so on) to every version automatically.
- Keep structured metadata for each version (file name, version number, storage path, size, and date and time of modification) in a MySQL database.
- Provide a version history from which any version can be selected and its details inspected.
- Allow two versions of a file to be compared and the differences identified.
- Allow an earlier version to be restored, and record every restoration in a recovery and audit log.

1.5 Importance of the Application

Because no version is ever overwritten, users are protected against accidental modification, replacement and deletion without having to create and manage backup copies by hand. The structured history makes it easy to see when a file changed, who changed it and how it changed, while one-click restoration reduces the time and effort needed to recover from a mistake. For students and academic teams, who frequently revise reports and project files, the system also provides a practical demonstration of version-control concepts in an everyday file management setting.

1.6 Target Users / Audience

The system is intended for general users who work with files and want a safe history of their changes, such as students, teachers, office staff and small teams, rather than only for software developers. Users manage their files, versions and recoveries through a web interface, and no knowledge of command-line tools is required.

CHAPTER 2: SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The application has modest hardware requirements, since the Java backend, the MySQL database and the local file storage can all run on a single computer. A development or demonstration machine with a dual-core processor, 4 GB of RAM and enough free disk space for the project, the MySQL data files and the stored file versions is sufficient. Because every version of a file is stored in full, at least 1 GB of free disk space is recommended.

2.2 Software Requirements

- Java Development Kit (JDK), to compile and run the backend
- MySQL Server (version 8.0 recommended)
- MySQL Workbench, to create, manage and test the database and SQL queries
- Visual Studio Code or IntelliJ IDEA, for development
- A modern web browser (Chrome, Edge or Firefox)
- Git, for tracking the project's own source code (optional)

2.3 Development Tools and Frameworks

The user interface is written in HTML, CSS and JavaScript. The server-side logic is written in Java, which handles file operations, processes requests and communicates with the MySQL database. MySQL stores file information, version details, timestamps, file paths and recovery status, while local file storage holds the actual uploaded files and their versions. Visual Studio Code and IntelliJ IDEA were used as development environments, and MySQL Workbench was used to design the schema and to test the SQL queries.

CHAPTER 3: TECHNOLOGY STACK

3.1 Frontend

The frontend is built with HTML, CSS and JavaScript. HTML defines the structure of the pages (login, dashboard, file list, upload form, version history, comparison and recovery screens), CSS provides a consistent and responsive layout, and JavaScript handles user interaction: drag-and-drop upload, search and filtering, display of the version timeline, and sending requests to the backend and showing the responses without reloading the whole page.

3.2 Backend

The backend is implemented in Java. It receives requests from the browser, stores uploaded files in local storage, generates the next version number, runs the file comparison logic, and reads and writes the metadata in MySQL. The code is organized so that request handling, business logic (versioning, comparison and recovery) and database access are kept in separate classes, which keeps each part easy to understand and test.

3.3 Database

MySQL stores the file registry (files), the version history (file_versions) and the recovery audit log (recovery_logs), together with a users table for ownership and requester identity. Relationships are expressed through foreign keys, and constraints such as primary keys, NOT NULL fields and a unique version number per file keep the stored data consistent. MySQL Workbench was used to create the tables and to test the SQL queries.

3.4 Version Control

The project's own source code can be tracked with Git during development. This is separate from the file versioning provided by the system itself: Git versions the project, while the File Versioning and Recovery System versions the files uploaded by its users.

3.5 Authentication and Security Libraries

Users sign in with a username or email and a password. Passwords are stored only as salted hashes, never as plain text. All SQL statements use parameterized queries (prepared statements) to prevent SQL injection. Uploaded files are saved under system-generated names inside a controlled storage folder, so a user-supplied file name can never be used to reach or overwrite other locations on the server. Restoration and permanent deletion require explicit confirmation, and every restoration is written to the audit log.

CHAPTER 4: SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram

The system follows a three-tier architecture with an additional local file store: the Presentation Layer (web browser), the Application Layer (Java backend) and the Data Layer (MySQL database and local file storage). Figure 4.1 shows these layers and how requests flow between them.

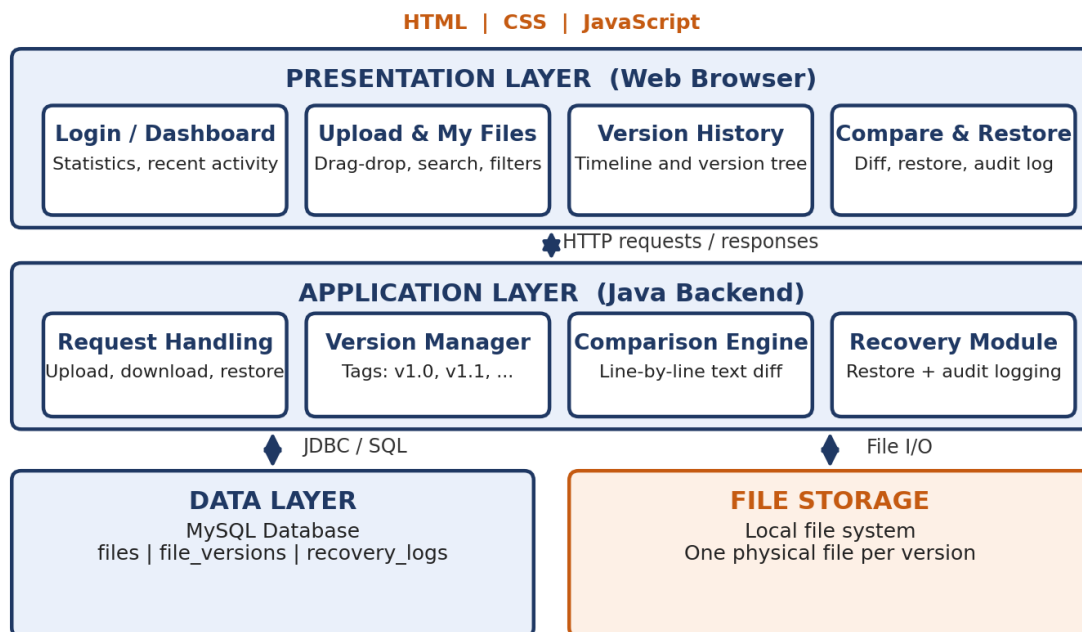


Figure 4.1: High-level architecture of the File Versioning and Recovery System

4.2 Description of Each Layer

The Presentation Layer is the web interface built with HTML, CSS and JavaScript. It never accesses the database or the file system directly; every action, such as uploading a file, requesting the version history, comparing two versions or restoring one, is sent to the Java backend as an HTTP request, and the response is displayed to the user.

The Application Layer is the Java backend. When a request arrives it is validated and handed to the appropriate module: the version manager assigns the next version tag and saves the physical file, the comparison engine compares two selected versions, and the recovery module restores a chosen version and records the action. The layer reads and writes metadata in MySQL and performs file input and output on the local storage.

The Data Layer has two parts. MySQL holds the indexed metadata (file records, version records with version number, storage path, size and timestamp, and recovery logs), while local file storage holds the physical files, with each version written to its own path so that no earlier content is overwritten. The database record and the physical file of a version are always created together.

4.3 Deployment Architecture

For development and demonstration, all components run on a single machine: the Java backend runs as a local server process, the MySQL server runs locally, the uploaded files and their versions are kept in a configured folder on the local disk, and the browser opens the web interface from the same machine. No cloud deployment or containerization was configured for this submission; Chapter 11 outlines how local storage could be replaced by cloud object storage.

CHAPTER 5: DESIGN

5.1 Database Design Principles

The schema was designed so that each fact is stored in exactly one place and every relationship between entities is expressed through a foreign key. The file registry holds only information about the file as a whole (name, owner, creation time and a pointer to the current version), while every version of that file has its own row in the version table. Integrity rules such as primary keys, foreign keys, NOT NULL fields and a unique combination of file and version number are enforced by the database itself, so invalid data is rejected even if the application code contains a bug.

5.2 ER Diagram / Database Schema

The database consists of the three core tables of the project design (files, file_versions and recovery_logs) plus a supporting users table, shown in Figure 5.1.

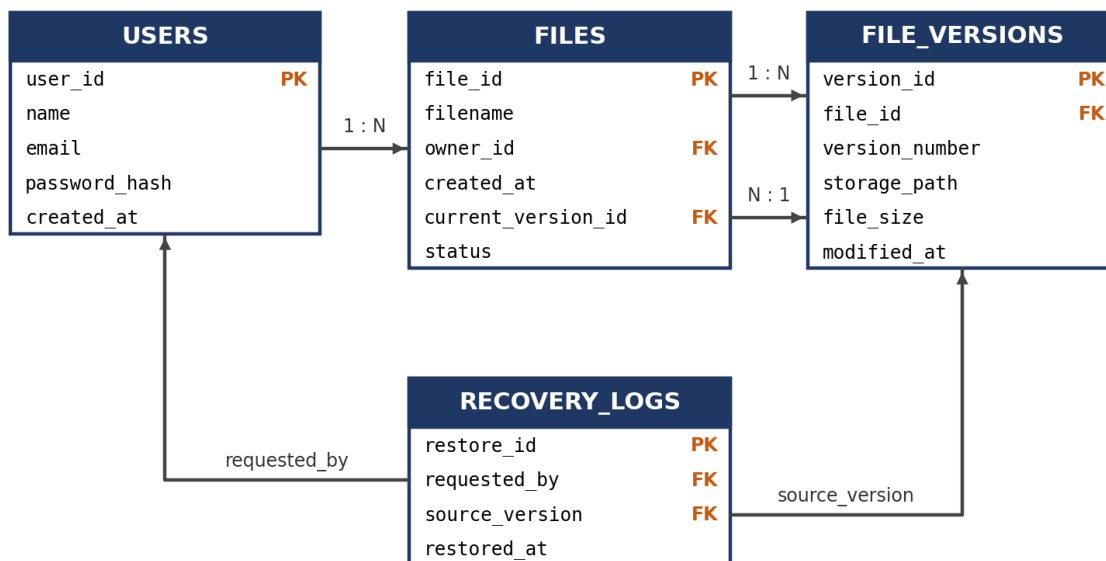


Figure 5.1: ER diagram / database schema

USERS stores the accounts that own files and request recoveries. **FILES** is the master registry, holding file_id, filename, owner_id, created_at and current_version_id, which points to the version that is currently active, together with a status field (Active or Archived). **FILE_VERSIONS** holds one row per saved version: version_id, file_id, version_number, storage_path, file_size and modified_at. **RECOVERY_LOGS** records each restoration through restore_id, requested_by, source_version and restored_at, which makes every recovery auditable.

5.3 Design Decisions and Assumptions

The following design decisions were taken explicitly and are recorded here rather than left implicit in the code:

- **No overwriting:** each upload or update creates a new row in file_versions and a new physical file; existing rows and files are never modified.

- **Current-version pointer:** FILES.current_version_id identifies the active version, so restoring an earlier version only moves this pointer and logs the action, and every other version remains available.
- **Archive instead of delete:** deleting a file sets its status to Archived and keeps its version history in the database so that the file can be recovered. Permanent deletion is a separate, confirmed action.
- **Supporting users table:** the owner_id and requested_by columns reference a users table so that every file and every recovery can be attributed to a person.
- **Sequential version tags:** versions of a file are numbered v1.0, v1.1, v1.2 and so on, in the order in which they are created.

CHAPTER 6: IMPLEMENTATION

6.1 Module-wise Implementation

The backend is organized into five modules: Authentication, File Management, Version Management, Comparison and Recovery. The most important logic is in the Version Management module. When a user uploads a new file or an updated copy of an existing file, the request is intercepted by the backend, which assigns the next sequential version tag (for example v1.0, then v1.1). Instead of replacing the existing file, the new content is written to a distinct file path in local storage. At the same time the backend registers an entry in MySQL containing the version ID, original file name, physical path, file size and timestamp, and updates the file's current-version pointer. Figure 6.1 shows this workflow.

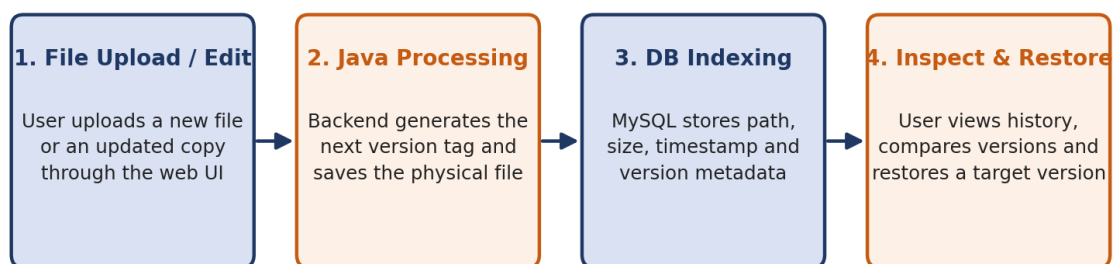


Figure 6.1: End-to-end workflow of version creation, inspection and restoration

The Comparison module takes two selected versions of a file, reads their stored content and, for text-based files, performs a line-by-line comparison to identify added, removed and modified lines. For file types that cannot be compared line by line, a metadata comparison (version number, size, modification time and file type) is shown instead, so the system never claims content differences that it has not actually calculated.

The Recovery module restores a selected earlier version. It first asks the user to confirm the action, then makes the selected version the active version of the file and writes an entry to the recovery log containing the restore ID, the requesting user, the source version and the time of restoration. No version is deleted by a restoration, so the full history is preserved. An archived file can likewise be restored from the archive, because its versions are still held in the database.

6.2 Front-end Logic

The frontend is written in plain HTML, CSS and JavaScript. Each screen has its own page or section, and shared elements such as the navigation sidebar, confirmation dialogs, status badges (Active, Archived, Current Version, Restored) and notification messages are written once and reused. JavaScript validates input before it is sent, shows upload progress, and updates tables and the version timeline when responses arrive. Destructive actions such as restoring a version or permanently deleting a file always show a confirmation dialog first.

6.3 API Integration

All communication between the browser and the Java backend takes place through HTTP requests. The JavaScript code keeps this request logic in one place, a small service layer with one

function per operation (upload file, list files, get version history, compare versions, restore version, list recovery logs), so that no page hardcodes a URL or repeats the same request code. Appendix D summarizes the operations.

6.4 Authentication and Authorization

A user must log in before using the system, and each user can see and manage only his or her own files. The backend checks the identity of the logged-in user on every request that reads or changes a file, and passwords are stored only as salted hashes. Restoring a version and permanently deleting a file require explicit confirmation, and every restoration is recorded in the audit log together with the identity of the user who requested it.

CHAPTER 7: FEATURES

7.1 List of Main Features

File management features:

- Upload a file by drag-and-drop or by browsing, with upload progress and automatic creation of version v1.0 for a new file
- Upload an updated copy of an existing file, which automatically creates the next version instead of overwriting the old one
- “My Files” page in grid or list view, with search, sorting and filtering by name, type, date and status
- File details page showing type, size, owner, created and last-modified dates, current version and number of versions
- Download the current version or any earlier version of a file
- Archive a file instead of deleting it, with a recycle-bin style Archived Files page and a separate, confirmed permanent-delete option

Versioning and recovery features:

- Version history shown as a visual timeline, with version number, file name, size, date and time, modified-by and status for each version
- Version tree view in which any version can be opened to see its details
- Side-by-side comparison of two versions, with added, removed and modified content highlighted for text files and a metadata comparison for other files
- One-click restoration of an earlier version, protected by a confirmation dialog
- Recovery and audit log listing every restoration (restore ID, file, source version, requested by, time and status)

Supporting features:

- Dashboard with statistics (total files, total versions, files modified today, archived files, recent recoveries and storage used), recent activity and recently modified files
- Storage overview showing the space used and the largest files
- Notifications for uploads, new versions, restorations and archiving, and a user profile page

7.2 How Each Feature Works

Automatic version creation: when a user uploads a new file, a file record is created and version v1.0 is stored. When the upload is an updated copy of an existing file, the next version number is generated (for example v1.3 becomes v1.4), the content is saved under a new storage path, a new row is added to file_versions and the file's current version is updated. A message such as “New version v1.4 created successfully” confirms the result. Figure 7.1 illustrates the resulting history of a file.

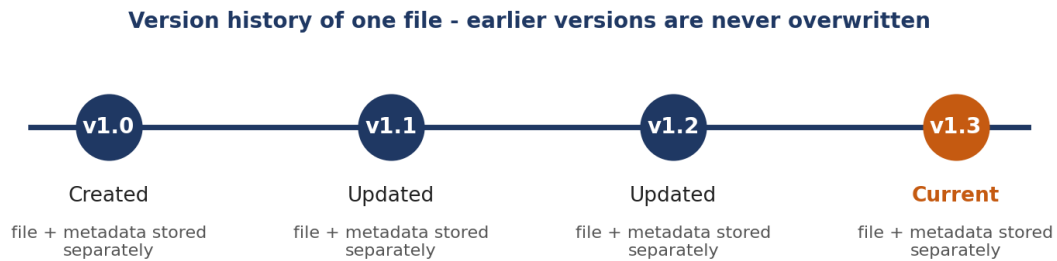


Figure 7.1: Version history of a file (earlier versions are never overwritten)

Version comparison: the user selects Version A and Version B of a file and clicks Compare Versions. For text files, the two stored contents are compared line by line and the result is displayed side by side with added, removed and modified lines highlighted, followed by a short difference summary. For other file types, a metadata comparison is shown and the interface states that the contents were not compared.

Recovery: the user selects an earlier version in the version history and chooses Restore this version. After the user confirms the dialog, the selected version becomes the current version, a success message is shown and a record is written to the recovery log. The other versions remain untouched.

Archive protection: when a user deletes a file, it is marked Archived rather than removed, and it moves to the Archived Files page. From there it can be restored with its full history intact, or permanently deleted after a second confirmation.

CHAPTER 8: TESTING

8.1 Postman API Testing

The operations exposed by the backend (upload, list files, version history, compare, restore, archive and recovery logs) can be tested directly with a REST client such as Postman, independently of the web interface. Requests are grouped by resource, and the response status and body are checked against the expected behavior for both valid and invalid inputs.

8.2 Integration Testing

Integration testing verifies that the browser, the Java backend, the MySQL database and the local file storage work correctly together. The central check is that a database record and a physical file always exist as a pair: after every upload, the new row in `file_versions` must have a matching file at its storage path, and after a restoration the current-version pointer and the recovery log must both reflect the action. The database tables are inspected directly in MySQL Workbench after each operation to confirm this.

8.3 Tools Used for Testing

- Postman (or a similar REST client): API testing of the backend operations
- MySQL Workbench: verifying table contents, constraints and SQL queries
- Web browser: manual testing of the interface (upload, history, comparison and recovery)
- File explorer: confirming that each version is stored as a separate physical file

8.4 Test Cases and Results

The test cases below define the expected behavior of the system. The Actual Result column is to be completed as each case is executed.

#	Test Case	Expected Result	Actual Result
1	Upload a new file	Version v1.0 created; file saved to storage; rows added to files and file_versions	Pending
2	Upload an updated copy of an existing file	Next version (for example v1.1) created; previous version remains unchanged	Pending
3	Upload several updates in sequence	Version numbers increase sequentially (v1.0, v1.1, v1.2 ...) with no gaps or duplicates	Pending
4	View the version history of a file	All versions listed with number, size, date and time; current version marked	Pending
5	Open the details of an earlier version	Correct metadata displayed for the selected version	Pending
6	Compare two versions of a text file	Added, removed and modified lines highlighted, with a difference summary	Pending
7	Compare two versions of a non-text file	Metadata comparison shown; no content differences claimed	Pending
8	Restore an earlier version after confirmation	Selected version becomes current; no version deleted; entry added to recovery_logs	Pending

#	Test Case	Expected Result	Actual Result
9	Cancel the restore confirmation dialog	No change to the current version; no log entry	Pending
10	Archive (delete) an active file	File marked Archived; version history retained in the database	Pending
11	Restore an archived file	File becomes Active again with its full version history	Pending
12	Permanently delete an archived file	Confirmation required; file and its versions removed only after confirming	Pending
13	Upload with no file selected	Request rejected with a clear error message	Pending
14	View the recovery and audit log	Every restoration listed with requester, source version and time	Pending

CHAPTER 9: DEPLOYMENT

9.1 Steps to Deploy

The application is deployed locally for development and demonstration as follows:

- Create the database and tables by running the SQL schema script in MySQL Workbench (a sample of the schema is given in Appendix A)
- Create the local folder in which uploaded files and their versions will be stored
- Set the database connection details and the storage folder path in the backend configuration
- Build and start the Java backend from IntelliJ IDEA or from the command line
- Open the web interface in a browser, log in and upload a first file to confirm that version v1.0 is created

9.2 Environment Configuration

Configuration values that differ between machines are kept outside the source code, in a configuration file: the MySQL host, port, database name, user name and password, and the path of the folder used for file storage. Credentials are never hard-coded in the program or committed to version control; a template file with placeholder values can be shared instead, so that the required settings are documented without exposing real passwords.

9.3 Hosting of Frontend and Backend

For this submission, the frontend, the backend, the database and the file storage all run on one local machine, which is appropriate for development and in-class demonstration. A production deployment was not required for this course submission. The same design could later be hosted on a server, with file storage moved to a cloud object-storage service as described in Chapter 11.

CHAPTER 10: CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

- **Avoiding overwriting:** ordinary file handling replaces old content, so the design had to ensure that every update is written to a new, distinct path.
- **Keeping the database and file system consistent:** a version consists of a database row and a physical file, and if only one of them were created the history would be incomplete or would point to a missing file.
- **Deciding what “restore” means:** whether to copy an old version forward as a new version or to make the old version active again, without losing any history.
- **Comparing files of different types:** only text-based files can be compared line by line; documents in binary formats cannot.
- **Storage growth:** because each version is stored in full, repeated updates to large files use disk space quickly.

10.2 Solutions Applied

To avoid overwriting, each version is saved under its own storage path and a new row is inserted in `file_versions`; existing rows and files are never updated.

To keep the database and file system consistent, the backend writes the physical file first and creates the database row immediately afterwards; if the database step fails, the newly written file is removed, so that no version is left half-created.

For restoration, the current-version pointer is moved to the chosen version instead of copying data. Nothing is lost, the action can be reversed by restoring a different version, and every restoration is logged.

For comparison, text files get a line-by-line comparison, while for all other types the interface shows a metadata comparison and states clearly that the contents were not compared. Storage usage is displayed on the dashboard so that users can see the space consumed, and delta compression is proposed as a future improvement.

10.3 Current Limitations or Known Bugs

- Every version is stored as a full copy, so storage use grows with each update (there is no delta compression yet).
- Files are stored on the local disk only; there is no cloud or remote backup, so a disk failure would lose both the files and their versions.
- Content comparison is available only for text-based files; other file types are compared by metadata only.
- No hash-based integrity check (such as SHA-256) is performed to detect a stored file that has been altered outside the system.
- Fine-grained role-based access control, for example restricting who may restore or permanently delete a version, is not yet implemented.
- The system has been run only as a single local deployment.

CHAPTER 11: FUTURE ENHANCEMENTS

11.1 Planned Features

- **Cloud object-storage sync:** extend local file storage to AWS S3 or Azure Blob Storage for high scalability and remote redundancy.
- **Delta compression:** store only the textual or binary differences between versions to reduce storage consumption.
- **Cryptographic integrity checks:** record a SHA-256 checksum for every stored version and verify it on download and restore.
- **Role-based access control (RBAC):** restrict who may restore, share or permanently delete versions.
- **Notifications:** email alerts for restorations and archived files, and a password-reset flow for user accounts.

11.2 Possible Integrations or Optimizations

- Using a diff library to provide richer comparison, such as word-level differences and comparison of common document formats.
- Adding retention policies that automatically remove or compress very old versions.
- Containerizing the application with Docker for a consistent, reproducible deployment environment.
- Adding an automated test suite (for example JUnit) to complement the manual testing, so that regressions are caught automatically.

CHAPTER 12: CONCLUSION

12.1 Summary of the Project

This project set out to replace the overwrite-on-save behavior of ordinary file managers with a system that keeps every version of a file and makes recovery simple. The File Versioning and Recovery System was designed with a web interface in HTML, CSS and JavaScript, a Java backend, a MySQL database for version metadata and recovery logs, and local file storage for the physical files.

12.2 What Was Achieved

The system stores each upload or update as a separate, numbered version; keeps a structured history with file name, version number, size and timestamp; lets users view and compare versions; restores earlier versions with a single confirmed action; protects against accidental deletion by archiving files instead of destroying them; and records every restoration in an audit log.

12.3 Skills Learned

Working on this project gave practical experience in relational database design (primary keys, foreign keys and constraints), writing SQL queries and connecting a Java application to MySQL, handling files in Java, designing version-numbering and recovery logic, building a responsive HTML, CSS and JavaScript interface, and planning systematic tests. It also provided a practical understanding of the ideas behind version control systems.

CHAPTER 13: REFERENCES

- Java Platform, Standard Edition — Official Documentation, <https://docs.oracle.com/en/java/>
- MySQL 8.0 Reference Manual, <https://dev.mysql.com/doc/refman/8.0/en/>
- MySQL Workbench Manual, <https://dev.mysql.com/doc/workbench/en/>
- MDN Web Docs — HTML, CSS and JavaScript reference, <https://developer.mozilla.org>
- IntelliJ IDEA — Documentation, <https://www.jetbrains.com/help/idea/>
- Visual Studio Code — Documentation, <https://code.visualstudio.com/docs>
- Git — Documentation, <https://git-scm.com/doc>
- Postman Learning Center, <https://learning.postman.com>

CHAPTER 14: APPENDICES

Appendix A: Sample Code Snippet

The SQL below creates the tables described in Chapter 5, and the query that follows returns the version history of a file.

```
CREATE TABLE users (
  user_id      INT AUTO_INCREMENT PRIMARY KEY,
  name         VARCHAR(100) NOT NULL,
  email        VARCHAR(150) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE files (
  file_id      INT AUTO_INCREMENT PRIMARY KEY,
  filename     VARCHAR(255) NOT NULL,
  owner_id     INT NOT NULL,
  created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  current_version_id INT NOT NULL,
  status       ENUM('Active', 'Archived') DEFAULT 'Active',
  FOREIGN KEY (owner_id) REFERENCES users(user_id)
);

CREATE TABLE file_versions (
  version_id   INT AUTO_INCREMENT PRIMARY KEY,
  file_id      INT NOT NULL,
  version_number VARCHAR(10) NOT NULL, -- v1.0, v1.1, ...
  storage_path VARCHAR(500) NOT NULL,
  file_size    BIGINT NOT NULL,
  modified_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE (file_id, version_number),
  FOREIGN KEY (file_id) REFERENCES files(file_id)
);

ALTER TABLE files
  ADD FOREIGN KEY (current_version_id) REFERENCES file_versions(version_id);

CREATE TABLE recovery_logs (
  restore_id   INT AUTO_INCREMENT PRIMARY KEY,
  requested_by INT NOT NULL,
  source_version INT NOT NULL,
  restored_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (requested_by) REFERENCES users(user_id),
  FOREIGN KEY (source_version) REFERENCES file_versions(version_id)
);

-- Version history of one file, newest first
SELECT version_number, file_size, modified_at
FROM   file_versions
WHERE  file_id = ?
ORDER BY modified_at DESC;
```

Appendix B: Installation and Setup Instructions

- Install the JDK, MySQL Server and MySQL Workbench.
- Run the schema script in MySQL Workbench to create the database and tables.
- Create the folder that will hold the uploaded files, and set its path and the database credentials in the backend configuration.
- Build and run the Java backend.
- Open the web interface in a browser.

Appendix C: Default Login Credentials

The demonstration accounts used for the project are listed below.

Account	Username / Email	Password
Sample user 1	[to be filled]	[to be filled]
Sample user 2	[to be filled]	[to be filled]

Appendix D: API Reference Summary

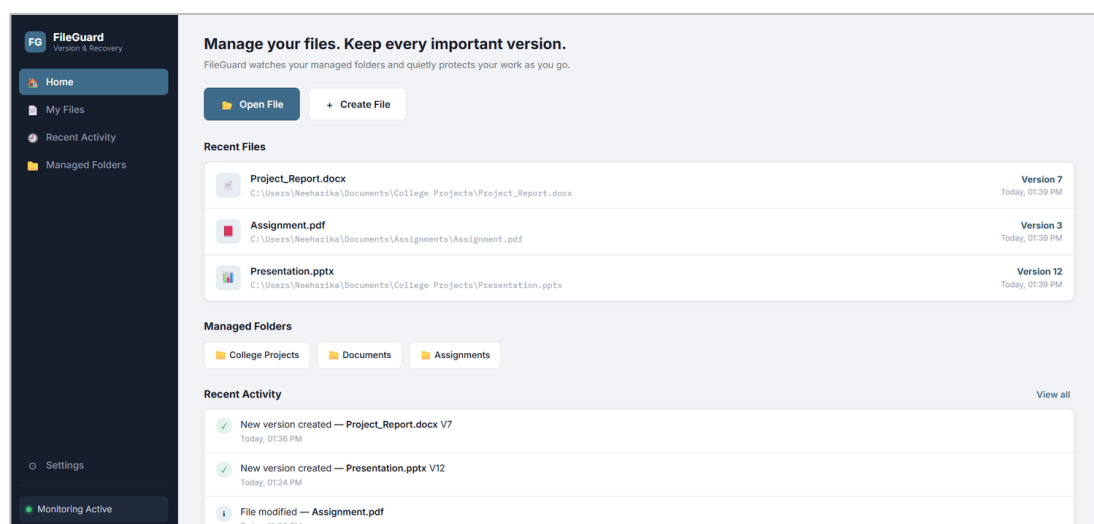
Conceptual summary of the operations used between the web interface and the Java backend (to be aligned with the final implementation).

Resource	Operations
Auth	POST /login, POST /logout
Files	GET /files, GET /files/{id}, POST /files/upload, PUT /files/{id}/archive, DELETE /files/{id}
Versions	GET /files/{id}/versions, POST /files/{id}/versions, GET /versions/{id}/download, GET /versions/compare
Recovery	POST /versions/{id}/restore, GET /recovery-logs
Storage	GET /storage/summary

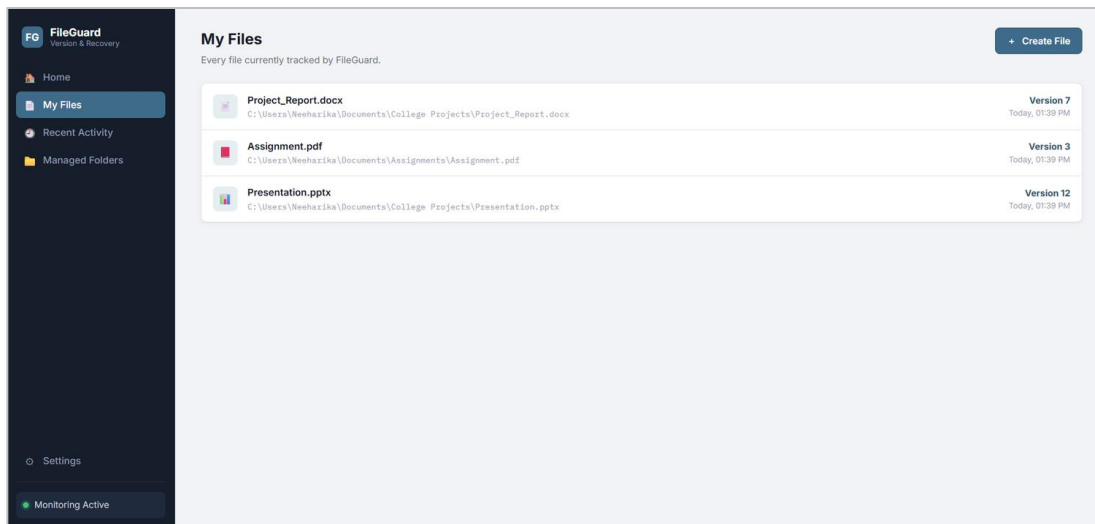
Appendix E: Screenshots

The following pages show screenshots taken from the running application. Screens for which no screenshot has been captured yet are marked with a placeholder to be completed before submission.

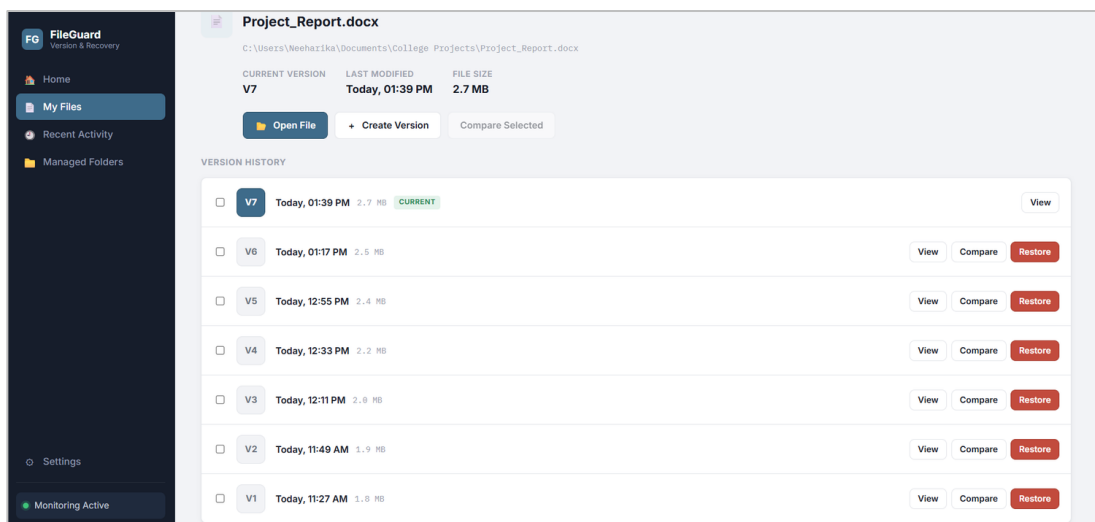
1. Dashboard



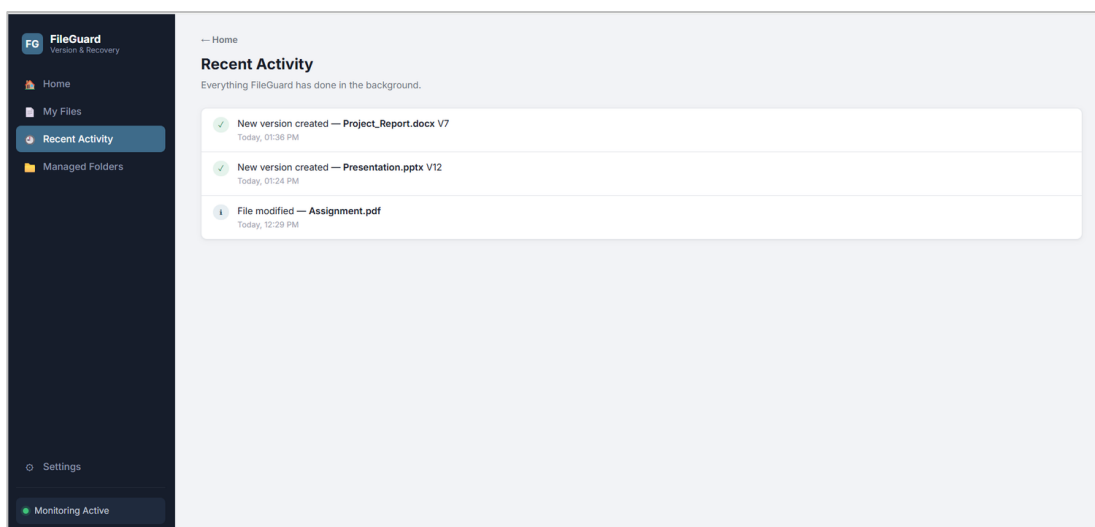
2. My Files page



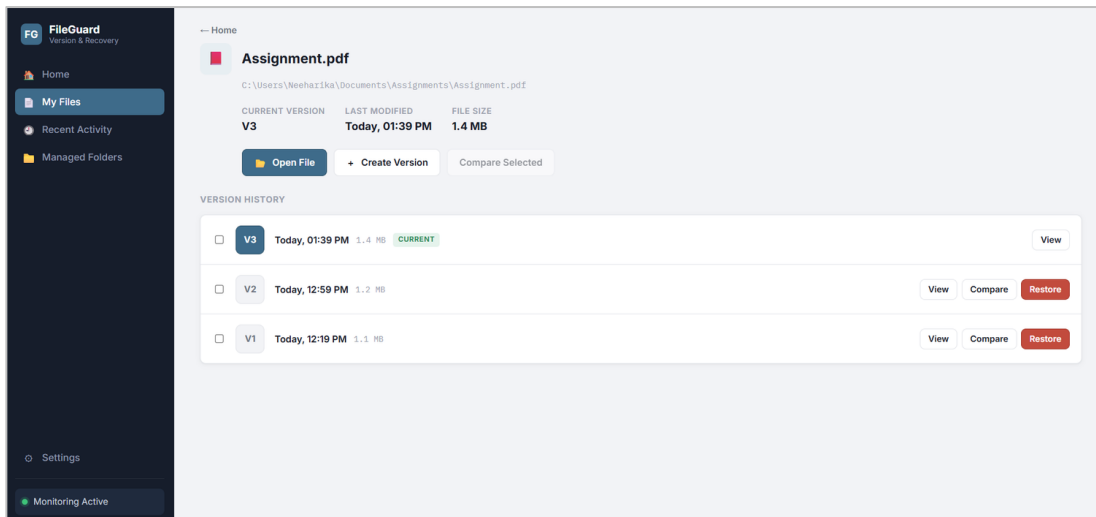
3. Version History (timeline)



4. Recovery & Audit Log



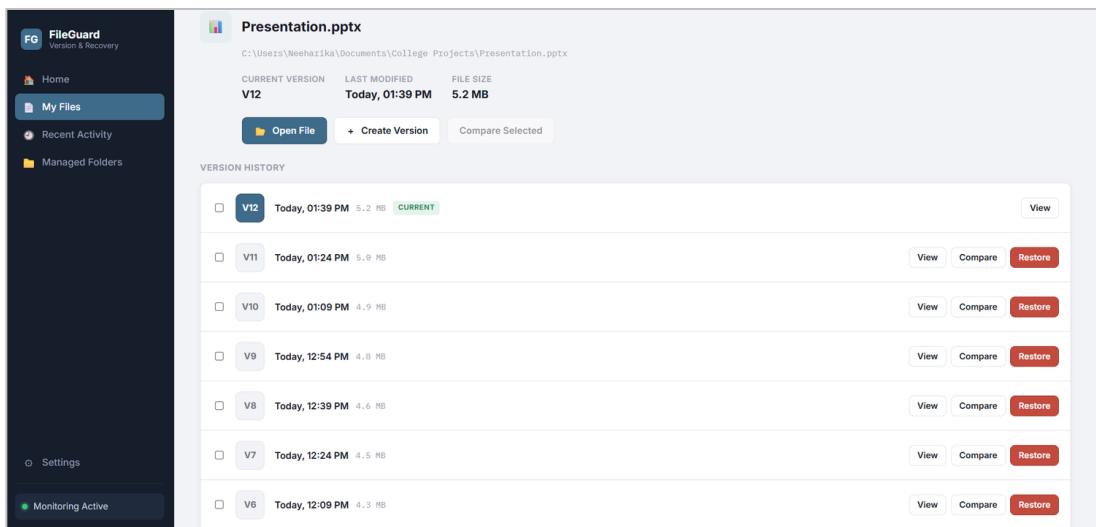
5. Version History – Assignment.pdf



The screenshot shows the FileGuard Version & Recovery interface for the file "Assignment.pdf". The left sidebar contains navigation options: Home, My Files, Recent Activity, Managed Folders, Settings, and Monitoring Active. The main content area displays the file's path, current version (V3), last modified time (Today, 01:39 PM), and file size (1.4 MB). Below this, the "VERSION HISTORY" section lists three versions: V3 (Today, 01:39 PM, 1.4 MB, CURRENT), V2 (Today, 12:59 PM, 1.2 MB), and V1 (Today, 12:19 PM, 1.1 MB). Each version has a checkbox, a "View" button, and "Compare" and "Restore" buttons.

Version	Last Modified	File Size	Status
V3	Today, 01:39 PM	1.4 MB	CURRENT
V2	Today, 12:59 PM	1.2 MB	
V1	Today, 12:19 PM	1.1 MB	

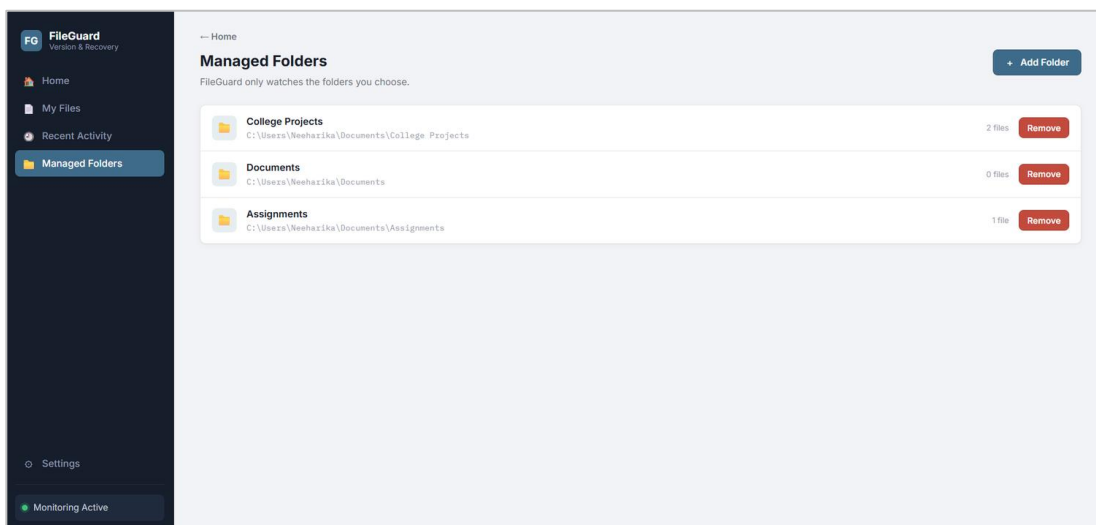
6. Version History – Presentation.pptx



The screenshot shows the FileGuard Version & Recovery interface for the file "Presentation.pptx". The left sidebar contains navigation options: Home, My Files, Recent Activity, Managed Folders, Settings, and Monitoring Active. The main content area displays the file's path, current version (V12), last modified time (Today, 01:39 PM), and file size (5.2 MB). Below this, the "VERSION HISTORY" section lists twelve versions from V12 down to V6. Each version has a checkbox, a "View" button, and "Compare" and "Restore" buttons.

Version	Last Modified	File Size	Status
V12	Today, 01:39 PM	5.2 MB	CURRENT
V11	Today, 01:24 PM	5.0 MB	
V10	Today, 01:09 PM	4.9 MB	
V9	Today, 12:54 PM	4.8 MB	
V8	Today, 12:39 PM	4.6 MB	
V7	Today, 12:24 PM	4.5 MB	
V6	Today, 12:09 PM	4.3 MB	

7. Managed Folders page



The screenshot shows the FileGuard Version & Recovery interface for the "Managed Folders" page. The left sidebar contains navigation options: Home, My Files, Recent Activity, Managed Folders, Settings, and Monitoring Active. The main content area displays the "Managed Folders" section with a "+ Add Folder" button. Below this, a list of managed folders is shown: "College Projects" (C:\Users\Neeharika\Documents\College Projects, 2 files), "Documents" (C:\Users\Neeharika\Documents, 0 files), and "Assignments" (C:\Users\Neeharika\Documents\Assignments, 1 file). Each folder has a "Remove" button.

Folder Name	Path	Files	Action
College Projects	C:\Users\Neeharika\Documents\College Projects	2 files	Remove
Documents	C:\Users\Neeharika\Documents	0 files	Remove
Assignments	C:\Users\Neeharika\Documents\Assignments	1 file	Remove

8. Settings page (monitoring and version retention)

